

Arrow on the Dotted Line

Seven waypoints. Dock → driveway → sidewalk gate → delivery → return → IRLock precision dock.
Every code path traced tick by tick.

7

WAYPOINTS

~104 m

OUTBOUND

96°

GATE TURN

9

CODE PHASES

250 ms

LOOP TICK

Arrow on the Dotted Line

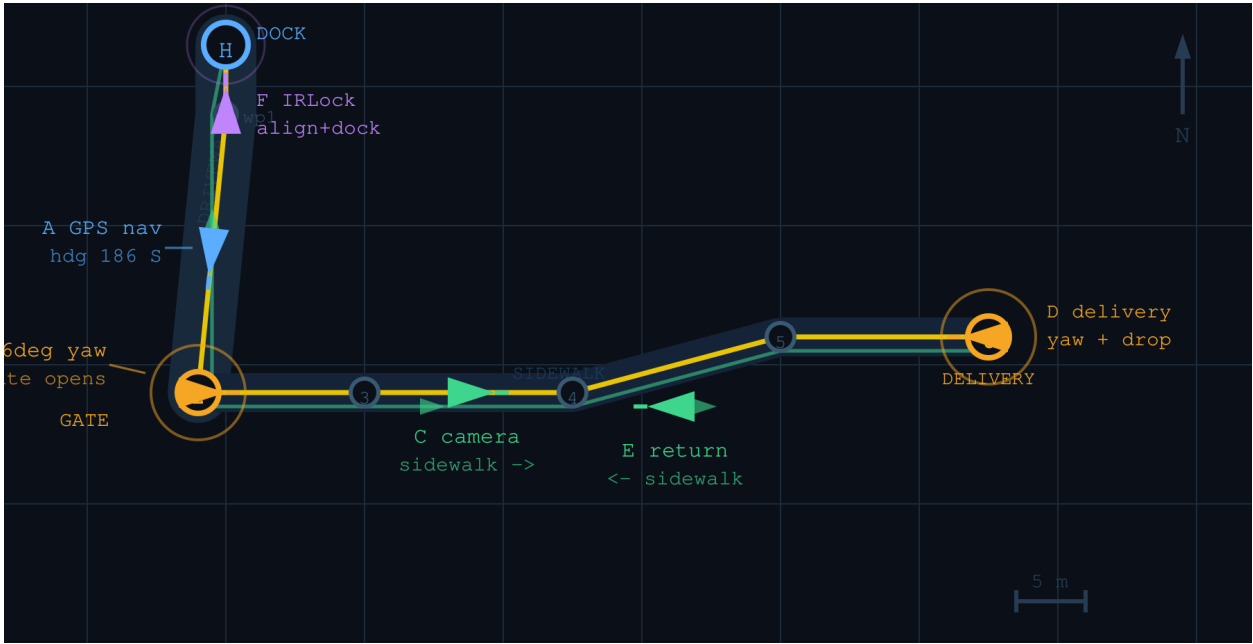
Seven waypoints. Dock → driveway → sidewalk gate → delivery → return → IRLock precision dock.
Every code path traced tick by tick.

ROUTE WAYPOINTS

SE Q	LATITUDE	LONGITUDE	DIST TO NEXT	ROLE
0	33.74900	-84.38800	2 m	Dock (skip)
1	33.74898	-84.38800	20 m S + 2 m W	Undock pos
2	33.74880	-84.38802	12 m E	GATE · 96° turn
3	33.74880	-84.38789	15 m E	Sidewalk
4	33.74880	-84.38773	15 m E + 4 m N	Sidewalk
5	33.74884	-84.38757	15 m E	Sidewalk
6	33.74884	-84.38741	—	Delivery

BIRD'S-EYE ROUTE MAP

5 px / m · North ↑ · East → · gold = outbound · green = return



- A – GPS nav (outbound)
- B – 96° yaw · gate opens
- C – sidewalk outbound

● D – delivery yaw · drop

● E – sidewalk return

● F – IRlock align · dock

Code Trace — Phase by Phase

① Driveway — GPS Navigation

WP0 → WP2

✓ on track

```
follow_sidewalk_enabled = false
// gate not open – skip FTYBR
nav_yaw = bearing_to_WP2 - heading
crosstrack = get_gps_crosstrack_bias_deg()
steering = nav_yaw x 0.4 + crosstrack
two_wheel Ackermann -> forward
```

```
sidewalk_follow_active
false
current_mission_seq
1 -> 2
yaw_to_waypoint
~ = 0 deg
max_steer
6° (clamped)
```

② Gate Arrival

WP2 arrival

■ yaw pending

```
is_sidewalk_gate_waypoint(seq=2)
// 96° turn >= 90° -> true
sidewalk_gate_pending = true
look_ahead WP3: bearing = 90° E
current heading = 186° S
yaw_error = -96°
needs_stop = |96°| > 90 -> true
advance seq -> 3, stop motors
```

```
sidewalk_gate_pending
true
sidewalk_follow_active
false
current_mission_seq
3
yaw_to_waypoint
-96°
```

③ Four-Wheel Yaw → Gate Opens

WP2, spinning

✓ gate opens

```
mission_yaw_abs = 96° > start(90°)
mission_yaw_should_run = true
steering_type -> four_wheels
yaw_white_rabbit(wr, -96°)
// spins CCW toward east
// yaw_abs <= 6° x 3 stable ticks:
four_wheels -> two_wheels
sidewalk_gate_pending -> false
sidewalk_follow_active = true
```

```
sidewalk_gate_pending
false
sidewalk_follow_active
true
heading
~ = 90° E
steering_type
two_wheels
edge_trail
started
```

④ Sidewalk — Camera Steering

WP3 → WP5

✓ camera guides

```
follow_sidewalk_enabled() = true
_yield_delivery = false // seq <= max
-> follow_the_yellow_brick_road(wr)
-> return

camera: edge 0.61 m ahead
steer 1.5 ft off left edge
FTYBR: seq++ when dist < 1m to WP
// advances WP3->4->5 internally
```

```
sidewalk_follow_active
true
seq advances via
FTYBR internal
camera edge
guiding
GPS / LiDAR
still active
```

⑤ Delivery — Option B Yield

WP6 → seq 7

✓ delivers

```
// FTYBR: within 1m of WP6
current_mission_seq++ -> 7
// next tick:
_yield_delivery: 7 > max(6) -> true
// FTYBR skipped, no return
GPS nav: seq=7 -> no waypoint
waypoint.latitude = null
-> yaw_for_package_delivery()
spin to WP5 bearing -> drop [check]
```

```
current_mission_seq
7 (past max)
_yield_delivery
true
finished_package_yaw
-> true
package_delivered
-> true
_post_delivery_yaw_carry
true
```

■ ■ ■ RETURN TRIP ■ ■ ■

⑥ Return — Sidewalk West

WP5 → WP2

✓ camera guides

```
arduino_msg: current_mission_seq -= 2 -> 5
package_delivered = true
follow_sidewalk_enabled() = true
_yield_delivery = false // delivered
-> follow_the_yellow_brick_road(wr)
-> return

FTYBR: seq-- when dist < 1m to WP
// reverse: WP5->4->3->gate(WP2)
```

```
package_delivered
true
sidewalk_follow_active
true
return_trail_reset
-> true (1st tick)
edge_trail
restarted
```

⑦ Gate Closes — GPS Home

WP2 → WP1

✓ on track

```
arrive WP2 (return): seq=2
is_sidewalk_gate_waypoint(2) = true
package_delivered = true
-> sidewalk_follow_active = false
// gate closes
follow_sidewalk_enabled() = false
GPS nav takes over
-> navigate to undock position (WP1)
nav_yaw + crosstrack -> two_wheel
```

```
sidewalk_follow_active
false
current_mission_seq
2 -> 1
nav mode
GPS two-wheel
target
undock lat/lng
```

⑧ Dock Approach — Heading Align

WP1 arrival

■ seeking
beacon

```
arrive undock pos: seq <= 1
-> dock_return_phase = 'align_heading'
// each tick:
_light_found = irlock.detected + is_fresh
if light found + |angle_x| <= 5°:
-> _begin_dock_handoff()
-> stop spin, phase = 'docking'
else:
yaw toward undock_heading
timeout 30s -> proceed blind
```

```
dock_return_phase
'align_heading'
undock_heading
recorded on depart
irlock.detected
polling
yaw_speed
proportional 10-18
```

⑨ IRLock — Precision Dock

dock_state ->
docked

[H] home

```
dock_return_phase = 'docking'  
setInterval(follow_the_light, 250)  
// separate 250ms loop – IRlock steers  
irlock.target.angle_x -> steer  
irlock.target.size -> throttle  
size >= size_stop_threshold:  
stop motors, dock_state = 'docked'  
run_mission sees dock_state = 'docked'  
-> clearInterval(mission_interval)  
-> mission complete
```

```
dock_return_phase  
'docking'  
follow_the_light  
running at 250ms  
steer src  
irlock.target.angle_x  
stop src  
irlock.target.size  
dock_state  
-> 'docked'  
mission_interval  
cleared
```

✓ Option B — Applied to run_mission.js

Inside the `follow_sidewalk_enabled` guard, a yield check detects when FTYBR has advanced `current_mission_seq` past the last waypoint (within 1 m of WP6). When true it skips FTYBR so the GPS nav else-branch reaches `yaw_for_package_delivery`. No changes to FTYBR, no changes to the delivery sequence itself.

```
// Yield delivery to GPS nav: FTYBR advances seq past max when Noah  
// arrives within 1m of the delivery WP. Drop through so the GPS nav  
// else-branch fires yaw_for_package_delivery.  
let _yield_delivery = false;  
if (!white_rabbit.mission.package_delivered) {  
  let _max_seq = 0;  
  for (let _i = 0; _i < white_rabbit.mission.waypoints.length; _i++) {  
    if (white_rabbit.mission.waypoints[_i].seq > _max_seq)  
      _max_seq = white_rabbit.mission.waypoints[_i].seq;  
  }  
  _yield_delivery = _max_seq > 0 &&  
    white_rabbit.mission.current_mission_seq > _max_seq;  
}  
if (!_yield_delivery) {  
  white_rabbit.follow_the_yellow_brick_road(white_rabbit);  
  return;  
}
```